

Task 2 – Student Profile Web Application

JavaScript Classes, Objects and DOM Manipulation

Task	Student Profile using JavaScript
File	index.html
Repository	full-stack-web-development
GitHub Folder	01-Observation/obsTask2_student_profile

1. Aim

To create a dynamic Student Profile web application using a JavaScript class, object creation, DOM manipulation, event handling, and basic CSS styling.

2. Task Statement

Create a JavaScript Student class with properties Name, Roll Number, Department and CGPA. Take values from the user, create a Student object, and dynamically generate the student profile on the webpage when the button is clicked. The application also validates CGPA so that it is between 3 and 10 with up to two decimal places.

3. Concepts Demonstrated

- JavaScript classes and objects
- DOM selection using `document.getElementById()`
- Creating HTML elements using `document.createElement()`
- Modifying content using `textContent`
- Adding elements using `appendChild()`
- Event handling using `addEventListener()`
- Input validation using `parseFloat()`, `isNaN()`, range checks and a regular expression
- Basic CSS styling of the generated profile

4. Program Working

1. The user enters Name, Roll Number, Department and CGPA.
2. The Create Profile button is selected from the DOM.
3. A click event is attached to the button.
4. The entered values are obtained from the input elements.
5. CGPA is checked for the range 3–10 and for at most two digits after the decimal point.
6. A Student object is created using the validated values.
7. The previous profile content is cleared.
8. New heading and paragraph elements are created dynamically.
9. The Student object's properties are placed into those elements using `textContent`.

10. The generated elements are added to the profile container using appendChild().

5. CGPA Validation

The application accepts CGPA values from 3 through 10. The regular expression `^\d+(\.\d{1,2})?$` allows an integer or a decimal value with one or two digits after the decimal point. The numeric checks reject values below 3 or above 10.

Input	Result
2.99	Invalid
3	Valid
8.42	Valid
10	Valid
10.01	Invalid
8.123	Invalid

6. Complete Source Code

The following is the final index.html submitted for the task.

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Profile</title>

  <style>
    body {
      font-family: Arial;
      text-align: center;
      background-color: #f2f2f2;
    }

    input {
      width: 250px;
      padding: 8px;
      margin: 5px;
      box-sizing: border-box;
    }

    button {
      padding: 10px 20px;
      margin: 10px;
      cursor: pointer;
    }

    #profile {
      width: 350px;
      margin: 20px auto;
      padding: 20px;
      background-color: white;
      border: 2px solid #333;
      border-radius: 10px;
      text-align: left;
    }

    #profile h2 {
      text-align: center;
    }
  </style>
</head>

<body>

  <h1>Student Profile</h1>

  <input type="text" id="name" placeholder="Enter Name">
  <br>

  <input type="text" id="roll" placeholder="Enter Roll Number">
  <br>

  <input type="text" id="department" placeholder="Enter Department">
  <br>

  <input
    type="number"
    id="cgpa"
    placeholder="Enter CGPA"
    min="3"
    max="10"
    step="0.01"
  >
  <br>

  <button id="createBtn">Create Profile</button>

  <div id="profile"></div>

  <script>

    // Student class
    class Student {

      constructor(name, roll, department, cgpa) {
        this.name = name;
        this.roll = roll;
        this.department = department;
        this.cgpa = cgpa;
      }
    }

    // Select button
    const button = document.getElementById("createBtn");
```

```

// Add click event
button.addEventListener("click", function() {

    // Get values from input fields
    const name = document.getElementById("name").value;
    const roll = document.getElementById("roll").value;
    const department = document.getElementById("department").value;
    const cgpa = document.getElementById("cgpa").value;

    // Validate CGPA
    const cgpaValue = parseFloat(cgpa);

    if (
        isNaN(cgpaValue) ||
        cgpaValue < 3 ||
        cgpaValue > 10 ||
        !/^d+(\.d{1,2})?$/i.test(cgpa)
    ) {
        alert("CGPA must be between 3 and 10 with up to 2 decimal places.");
        return;
    }

    // Create Student object
    const student = new Student(
        name,
        roll,
        department,
        cgpa
    );

    // Select profile container
    const profile = document.getElementById("profile");

    // Clear previous profile
    profile.innerHTML = "";

    // Create HTML elements dynamically
    const heading = document.createElement("h2");
    const namePara = document.createElement("p");
    const rollPara = document.createElement("p");
    const departmentPara = document.createElement("p");
    const cgpaPara = document.createElement("p");

    // Add content
    heading.textContent = "Student Profile";
    namePara.textContent = "Name : " + student.name;
    rollPara.textContent = "Roll No : " + student.roll;
    departmentPara.textContent =
        "Department : " + student.department;
    cgpaPara.textContent = "CGPA : " + student.cgpa;

    // Add elements to webpage
    profile.appendChild(heading);
    profile.appendChild(namePara);
    profile.appendChild(rollPara);
    profile.appendChild(departmentPara);
    profile.appendChild(cgpaPara);

});

</script>

</body>
</html>

```

7. Expected Output

After entering valid values and clicking **Create Profile**, the profile is generated dynamically on the webpage.

Student Profile	
Name	Ravi Kumar
Roll No	A23126552001
Department	CSE (AI & ML)
CGPA	8.42

8. Key Code Explanation

class Student — Defines the Student class and its four properties. The constructor assigns the values supplied when an object is created.

document.getElementById() — Selects existing HTML elements such as the input fields, button and profile container.

addEventListener('click', ...) — Runs the specified function whenever the Create Profile button is clicked.

new Student(...) — Creates an object of the Student class using the values entered by the user.

document.createElement() — Creates new h2 and p elements dynamically through JavaScript.

textContent — Places text into the dynamically created elements.

appendChild() — Adds the newly created elements to the profile container in the webpage.

9. Learning Outcomes

- Understand how JavaScript classes and constructors are used.
- Create objects from a class using user-provided values.
- Select and manipulate DOM elements.
- Create and insert HTML elements dynamically.
- Handle user actions with event listeners.
- Perform client-side input validation.
- Apply CSS styling to dynamically generated content.

10. Viva Questions and Answers

Q: What is a class in JavaScript?

A: A class is a template for creating objects. It defines properties and methods that objects created from it can use.

Q: What is an object?

A: An object is an instance of a class containing actual values for the properties defined by that class.

Q: Why is new Student(...) used?

A: It creates a new Student object and calls the Student constructor with the supplied values.

Q: What is DOM manipulation?

A: DOM manipulation means using JavaScript to select, create, modify or remove elements in an HTML document.

Q: Why is addEventListener() used?

A: It attaches an event handler to an element. Here it runs the profile-generation code when the button is clicked.

Q: What does createElement() do?

A: It creates a new HTML element in memory, such as an h2 or p element, which can then be added to the webpage.

Q: Why is.textContent used?

A: It sets the text content of an HTML element without interpreting the supplied text as HTML.

Q: What does appendChild() do?

A: It adds a newly created node as a child of a selected parent element.

Q: Why is parseFloat() used for CGPA?

A: It converts the input string into a floating-point number so numeric range comparisons can be performed.

Q: Why is the regular expression used?

A: It ensures the entered CGPA has no more than two digits after the decimal point.

11. GitHub Submission

The final source file is uploaded to the GitHub repository under the 01-Observation/obsTask2_student_profile folder.

GitHub:

https://github.com/venugopalreddychittela/full-stack-web-development/tree/main/01-Observation/obsTask2_student_profile